`Orbit`

Continuous Navigation via Actor-Critic
Methods using Neural Function Approximation

Stochastic Batman

March 16, 2026

Introduction

This document outlines the thinking process and systematic design for `Orbit`. It serves as a bridge between the theoretical foundations established in *Mathematical Foundations of Reinforcement Learning* by Shiyu Zhao and the practical implementation of neural function approximation in a continuous coordinate space.

1 Environment Specification (`env.rs`)

The environment is modeled as a Markov Decision Process (MDP) where the agent must learn to navigate toward a target in a bounded 2D plane.

1.1 State Space $\mathcal{S}$

The state space is defined over a four-dimensional hypercube. To facilitate stable training in neural networks, all coordinates are normalized to the range $[-1, 1]$. I define the state vector $s \in \mathcal{S}$ as:

$$s \doteq [x, y, x_{target}, y_{target}]^\top \in [-1, 1]^4$$

To maintain consistency in subsequent notation, I denote the components of any state s as:

- $s[0] = x$ (Agent's horizontal position) and $s[1] = y$ (Agent's vertical position)
- $s[2] = x_{target}$ (Target's horizontal position) and $s[3] = y_{target}$ (Target's vertical position)

Notation Note: I use the shorthand $s[i, \ldots, k]$ to represent a sub-vector consisting of elements from index i to k inclusive. For example, $s[0, 1]$ refers to the agent's coordinate pair $[s[0], s[1]]^\top$.

1.2 Action Space $\mathcal{A}$ and Transition Dynamics

I employ a discrete action space $\mathcal{A} = \{a_1, a_2, a_3, a_4, a_5\}$. Each action corresponds to a displacement vector $\Delta a \in \mathbb{R}^2$ with a step size magnitude $\Delta = 0.05$. The physical directions follow the clockwise convention:

$$\mathcal{A} = \{ \overset{\text{North}}{a_1} , \overset{\text{East}}{a_2} , \overset{\text{South}}{a_3} , \overset{\text{West}}{a_4} , \overset{\text{Stay}}{a_5} \}$$

The state transition probability $p(s'|s,a)$ is deterministic. The next state s' is calculated by applying the displacement to the agent's coordinates $s[0,1]$ and clipping the result to the boundary:

$$s'[i] = \text{clip}(s[i] + \Delta a_i, -1, 1),\ i \in \{0,1\}$$

The target coordinates remain stationary: $s'[2,3] = s[2,3]$.

1.3 Reward Function $r(s,a)$

The reward signal provides dense feedback based on the Euclidean distance between the agent and the target. I define the distance function between any two points p_1, p_2 as:

$$\text{dist}(p_1, p_2) \doteq \sqrt{(p_1[0] - p_2[0])^2 + (p_1[1] - p_2[1])^2}$$

The reward function is formulated to ensure the boundary penalty is strictly lower than any possible reward achievable within the valid state space. In the square $[-1,1]^2$, the maximum possible distance (the diagonal) is $\sqrt{2^2+2^2} = \sqrt{8} \approx 2.8284$.

I define the penalty as -2.85. This extra ≈ -0.02 is to ensure that a boundary collision is always perceived by the Critic as a "worse" state than being at the furthest possible valid coordinate from the target.

$$r(s,a) = \begin{cases} -2.85 & \text{if } s[0,1] + \Delta a \notin [-1,1]^2 \text{ (Boundary Crossing)} \\ -\text{dist}(s'[0,1], s'[2,3]) & \text{otherwise} \end{cases}$$

1.4 Termination and Episodes

The task is episodic with the following conditions:

- **Success Condition:** The episode terminates when $\text{dist}(s[0,1], s[2,3]) < 0.05$.
- **Truncation Condition:** An episode is capped at $T_{max} = 250$ steps.
- **Terminal Value:** Following the book's convention, the value of the terminal state $v(s_{terminal})$ is defined as 0.

2 Neural Function Approximation (`model.rs`)

Following the Actor-Critic framework (Zhao, Chapter 10), I approximate the policy π and the state-value function v using neural networks.

2.1 Network Architectures

I define two separate Multi-Layer Perceptrons (MLPs) with parameters θ (Actor) and w (Critic). Both networks receive the state vector $s \in [-1,1]^4$ as input.

- **Actor Network** $\pi(a|s,\theta)$: Maps s to 5 logits, followed by a softmax layer to produce a probability distribution over $\mathcal{A} = \{a_1, \ldots, a_5\}$.
- **Critic Network** $\hat{v}(s,w)$: Maps s to a single scalar representing the state-value $v_\pi(s) = \mathbb{E}[G_t|S_t = s]$, which is the expected discounted return from state s.

2.2 The TD Error and Objective Functions

To train the networks, I must derive the update signals based on the environment's feedback.

2.2.1 Original Equations

From the textbook, the Temporal Difference (TD) error is $\delta_t = R_{t+1} + \gamma v(S_{t+1}) - v(S_t)$. The objective is to minimize the Value Error for the Critic and maximize the expected return for the Actor via the Policy Gradient Theorem:

$$\nabla_\theta J(\theta) = \mathbb{E}[\nabla_\theta \ln \pi(A_t|S_t, \theta) q_\pi(S_t, A_t)]$$

2.2.2 Derivations for `Orbit`

In my implementation, I use the TD error δ_t as an unbiased estimate of the **advantage signal** $A(s, a) \doteq q_\pi(s, a) - v_\pi(s)$, which measures the relative benefit of a specific action compared to the average value of the state. I derive the following specific equations:

1. **TD Error with Termination Logic**: For a transition yielding reward R_{t+1} and next state S_{t+1}:

$$\delta_t = \begin{cases} R_{t+1} - \hat{v}(S_t, w) & \text{if } \texttt{is\_done = true} \\ R_{t+1} + \gamma \hat{v}(S_{t+1}, w) - \hat{v}(S_t, w) & \text{otherwise} \end{cases}$$

   Note: If `is_truncated = true`, I still bootstrap with $\gamma \hat{v}(S_{t+1}, w)$ as the truncation is a boundary of time, not the underlying MDP state.

2. **Critic Loss (L_w)**: Minimize the Mean Squared Error (MSE) of the TD error:

$$L(w) = \frac{1}{2}\delta_t^2$$

3. **Actor Loss (L_θ)**: Using the log-likelihood trick and the TD error as the advantage signal, I define the loss for the `dfdx` optimizer as:

$$L(\theta) = -\ln \pi(A_t|S_t, \theta)\delta_{t,\text{stop\_grad}}$$

   where $\delta_{t,\text{stop\_grad}}$ indicates that the TD error is treated as a constant scalar during the actor's backpropagation. "$-$" before ln is because `dfdx` minimizes instead of maximizing.

2.3 Optimization and Hyperparameters

The parameters θ and w are updated using the Adam optimizer. To ensure stable convergence, I employ a two-time-scale update rule where the Critic typically learns faster than the Actor to provide a stable baseline for the advantage signal:

- **Discount Factor (γ)**: 0.99 (Prioritizing long-term navigation).
- **Learning Rates**: $\alpha_\theta = 1 \times 10^{-4}$ (Actor), $\alpha_w = 1 \times 10^{-3}$ (Critic).
- **Entropy Regularization**: To prevent premature convergence to a sub-optimal deterministic policy, I may add an entropy term $H(\pi)$ to the Actor loss: $L_{total}(\theta) = L(\theta) - \beta H(\pi(s, \theta))$ where $\beta = 0.01$.

2.4 Softmax Policy Derivation

For the discrete action set $\{a_1, \ldots, a_5\}$, the probability of selecting action a_i given logits z from the Actor is:

$$\pi(a_i|s, \theta) = \frac{e^{z_i(s,\theta)}}{\sum_{j=1}^{5} e^{z_j(s,\theta)}}$$